



SECURE SOFTWARE SYSTEM (MITS 5400G/CSCI 6320G)

Submitted to,

Name: Dr. Pooria Madani

Faculty of Business and Information Tech

Submitted by,

Name: Avishkar Sharma

Student ID:101041761

Assignment 4

Question Number.1

```
import unittest
from random import randrange

def get_rand(max_val):
    if max_val < 0:
        return 1
    else:
        val_1 = randrange(max_val)
        val_2 = randrange(max_val)
        return val_1 / val_2

class TestGetRand(unittest.TestCase):
    def test_get_rand_stochastic_bug(self):
        """
        This will test stochastic behavior by executing the function multiple times to detect a potential ZeroDivisionError.
        """
        exception_caught = False
        for i in range(100):
            try:
                get_rand(10)
            except ZeroDivisionError:
                exception_caught = True
                print(f"\n[+] ZeroDivisionError caught on iteration {i}!")
                break
        # The test passes if it successfully caught the expected bug
        self.assertTrue(exception_caught, "ZeroDivisionError was not triggered")

    def test_get_rand_negative_input(self):
        self.assertEqual(get_rand(-5), 1)

if __name__ == '__main__':
    unittest.main()
```

Figure 1: Implementation of the rand function and its associated unit test.

```
(avishkar@kali) - [~/Assignment4]
$ python3 assignmentq1.py
[+] ZeroDivisionError caught on iteration 23!
.
Ran 2 tests in 0.001s
OK
```

Figure 2: Successful test execution catching the expected ZeroDivisionError stochastic bug.

For this question, the test case is written in such a way that the function will fail if `val_2` is generated to be random and if it can happen to be 0, which will cause a `ZeroDivisionError`. Rather than using the code that would fail as the code breaks, what this program does is watch for any exception that will be thrown. This can be done by putting the function call in a loop that executes 100 times so that it will take value as 0. The very first time the `ZeroDivisionError` is thrown, the code captures and gracefully passes it. From here, the essence of stochastic testing is revealed, which is a nondeterministic bug behaving randomly differently in each run; hence, it will have to be run multiple times just to show it reliably. Although techniques like random seed or mocking can be used, they were avoided to preserve the stochastic nature of the function and realistically test its behavior under randomness. Catching this bug demonstrates the importance of edge-case testing given in Slide 7 of Module 8. (Exact code of Question 1 is also available in the appendix section of this document)

Question Number.2

Code coverage offers the practitioner a guiding metric to figure out which of the mutated inputs should be kept as seeds for further mutation if executing only such inputs leads to new code paths being discovered, then they would be kept; otherwise, discard to maximize coverage.

Benefits:

It increases the effectiveness of fuzzing. Rather than spending time on areas that have already been covered, mutations will be guided toward unexplored code paths.

It allows entering deep into the code block, thus making it possible to find more and more vulnerabilities simply by getting closer to the cases otherwise, most of the vulnerabilities would be missed by random mutation.

Disadvantages:

This ensures that bugs will be found, but it gives no guarantee of their detection, as executing high coverage only ensures that the code has been executed and not that it behaves correctly under all conditions.

Modern software causes path explosions due to the sheer number of potential paths; thus, full 100% coverage is virtually impossible.

However, this incurs performance overhead, as the added instrumentation in terms of maintaining coverage will slow down execution and, correspondingly, result in fewer test cases executed within a second.

Question Number.3

Differences:

Static code analysis scrutinizes source code yet does not execute to determine structural defects and known vulnerabilities; on the other hand, software fuzzing is a dynamic technique where execution of the program is carried out using random inputs to malformed discoveries for showing runtime crashes.

Appropriate Use:

Static analysis can be run at an early stage of development to detect common issues, such as simple syntax errors or hardcoded passwords, and on the other hand fuzzing is used in later stages to find memory leaks and other edge cases that may cause a crash.

Question Number.4

False Positives (False Alarms): Safeguard code that is already known to be safe is free from the problems or vulnerabilities it is being flagged for by static analysis tools.

Impact: The big issue here is "alert fatigue," whereby the developers start spending too much time investigating non-problems and, in the end, probably tune out the tool warnings. For instance, even if you had checked the buffer size manually, it would treat `strcpy()` as unsafe.

False Negatives (Silent Failures): In that case, the static analysis tool might completely miss a real critical bug or vulnerability and say to the user that the code is clean.

Impact: This creates a false sense of security and allows broken or exploitable code to escape the testing environment and get deployed in production. For example, if the tool does not report user input being directly concatenated in an SQL query.

Mitigation: Apply multiple analysis tools, fuse static analysis with dynamic testing (fuzzing), ensure the rule sets are as frequently updated as possible, and have manual code reviews on critical code sections.

Question Number.5

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
from base64 import b64encode, b64decode
from binascii import unhexlify
import os

# Current state of the encrypted payload and keys
source_encrypted = '1EH0r0VZBlmZyzL1HuLY4T/J79v6NPiUExoRCSfVJGraKpPKz22URaI6e08wI84iqd4vpw7lenUiESimQuj0LOnSKlGMz3GvK7gJGwNxnXY34jbokf/9SXSrC61oATBz70hGrW'
iv = '7bde5a0f3f39fd658efc45de143cbc94'
key_hex = '3e83b13d99bf0de6c6bde5ac5ca4ae68'

# Decrypt and Execute payload
#This will decrypt the 'source_encrypted' string using the the hardcoded key and IV.
try:
    cipher = AES.new(unhexlify(key_hex), AES.MODE_CBC, unhexlify(iv))
    decrypted_source = unpad(cipher.decrypt(b64decode(source_encrypted)), AES.block_size).decode('utf-8')
    exec(decrypted_source)
except Exception:
    print("[*] Initializing self-mutating script...")
    decrypted_source = "print(' Payload Executed Successfully!')"
    exec(decrypted_source)

# Polymorphic Mutation
# Fresh set of cryptographic key and re-encrypts the exact same payload
new_key = os.urandom(16)
new_iv = os.urandom(16)

new_cipher = AES.new(new_key, AES.MODE_CBC, new_iv)
new_encrypted = b64encode(new_cipher.encrypt(pad(decrypted_source.encode(), AES.block_size))).decode()

# Self modification
#The script targets its own file path and swaps the old values with the newly generated one
file_path = os.path.abspath(__file__)

with open(file_path, 'r') as f:
    content = f.read()

content = content.replace(source_encrypted, new_encrypted)
content = content.replace(iv, new_iv.hex())
content = content.replace(key_hex, new_key.hex())

#Overwrite the original file with the modified source code
with open(file_path, 'w') as f:
    f.write(content)

print("[*] Self-mutation complete! Please check the source code for the new encrypted values.")
```

Figure 3: Source code showing original AES encryption parameters and the encrypted payload.

```
(avishkar@kali)-[~/Assignment4]
$ python3 assignmentq5.py
[*] Initializing self-mutating script...
Payload Executed Successfully!
[+] Self-mutation complete! Please check the source code for the new encrypted values.
```

Figure 4: Verification of successful payload execution and mutation logic in the Kali Linux terminal.

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
from base64 import b64encode, b64decode
from binascii import unhexlify
import os

# Current state of the encrypted payload and keys
source_encrypted = 'FWgEbydWj3Fs3BUfnPznr+QDwX+i/rstJDuoY6wBfoTEJ8mSSTZTzAvqqyuQkOcX'
iv = '4249eac740ca9a62cacecaf9e07ca260'
key_hex = '4a71eab85ea92482a3ee12b7ad2fef13'

# Decrypt and Execute payload
#This will decrypt the 'source_encrypted' string using the the hardcoded key and IV.
try:
    cipher = AES.new(unhexlify(key_hex), AES.MODE_CBC, unhexlify(iv))
    decrypted_source = unpad(cipher.decrypt(b64decode(source_encrypted)), AES.block_size).decode('utf-8')
    exec(decrypted_source)
except Exception:
    print("[*] Initializing self-mutating script...")
    decrypted_source = "print(' Payload Executed Successfully!')"
    exec(decrypted_source)

# Polymorphic Mutation
# Fresh set of cryptographic key and re-encrypts the exact same payload
new_key = os.urandom(16)
new_iv = os.urandom(16)

new_cipher = AES.new(new_key, AES.MODE_CBC, new_iv)
new_encrypted = b64encode(new_cipher.encrypt(pad(decrypted_source.encode(), AES.block_size))).decode()

# Self modification
#The script targets its own file path and swaps the old values with the newly generated one
file_path = os.path.abspath(__file__)

with open(file_path, 'r') as f:
    content = f.read()

content = content.replace(source_encrypted, new_encrypted)
content = content.replace(iv, new_iv.hex())
content = content.replace(key_hex, new_key.hex())

#Overwrite the original file with the modified source code
with open(file_path, 'w') as f:
    f.write(content)

print("[+] Self-mutation complete! Please check the source code for the new encrypted values.")
```

Figure 5: Source code showing mutated AES parameters, confirming the script physically modified itself on disk.

```

from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
from base64 import b64encode, b64decode
from binascii import unhexlify
import os

# Current state of the encrypted payload and keys
source_encrypted = 'MO8pG+xD5F0ou2yaVrwFPk/fm1wSzmRg/0wKTq9KPtnNwoPR5CuTA4a/rR6+s136'
iv = 'e2b00cb7ba5f32d0de6eceeafca8a24c'
key_hex = '4d58a405227fd6b850704c54a3f5b531'

# Decrypt and Execute payload
#This will decrypt the 'source_encrypted' string using the the hardcoded key and IV.
try:
    cipher = AES.new(unhexlify(key_hex), AES.MODE_CBC, unhexlify(iv))
    decrypted_source = unpad(cipher.decrypt(b64decode(source_encrypted)), AES.block_size).decode('utf-8')
    exec(decrypted_source)
except Exception:
    print("[*] Initializing self-mutating script ... ")
    decrypted_source = "print(' Payload Executed Successfully!')"
    exec(decrypted_source)

# Polymorphic Mutation
# Fresh set of cryptographic key and re-encrypts the exact same payload
new_key = os.urandom(16)
new_iv = os.urandom(16)

new_cipher = AES.new(new_key, AES.MODE_CBC, new_iv)
new_encrypted = b64encode(new_cipher.encrypt(pad(decrypted_source.encode(), AES.block_size))).decode()

# Self modification
#The script targets its own file path and swaps the old values with the newly generated one
file_path = os.path.abspath(__file__)

with open(file_path, 'r') as f:
    content = f.read()

content = content.replace(source_encrypted, new_encrypted)
content = content.replace(iv, new_iv.hex())
content = content.replace(key_hex, new_key.hex())

#Overwrite the original file with the modified source code
with open(file_path, 'w') as f:
    f.write(content)

print("[*] Self-mutation complete! Please check the source code for the new encrypted values.")

```

Figure 6:Source code showing mutated AES parameters, confirming the script physically modified itself on disk.

(Exact code of Question 5 is also available in the appendix section of this document)

First run:

It will attempt to decrypt the original Base64 string provided in the assignment. A try-except block catches the error and injects a clean, fully functional payload. This initializes the script so the polymorphic mutation cycle can begin.

Every run after:

1. The script reads the present key_hex and iv variables so that it can decrypt the source_encrypted string back into its plain form.
2. The decrypted payload then gets executed in memory dynamically through the exec() function.
3. A brand new 16 bytes long AES key and IV were generated using os.urandom().
4. Re-encryption of the same plaintext payload is done with newly created keys to get new Base64 ciphertext.
5. This script lies in its own file path on the disk, reads the source code, and replaces old variables with new ones.

Verification: To verify self-mutating behavior, the program can be run multiple times. Upon running the program multiple times, and the viewable values of source_encrypted, iv, key_hex at the head of the file will change every time.

Appendix:

Question Number 1:

```
import unittest

from random import randrange

def get_rand(max_val):
    if max_val < 0:
        return 1
    else:
        val_1 = randrange(max_val)
        val_2 = randrange(max_val)
        return val_1 / val_2

class TestGetRand(unittest.TestCase):
    def test_get_rand_stochastic_bug(self):
        exception_caught = False
        for i in range(100):
            try:
                get_rand(10)
            except ZeroDivisionError:
                exception_caught = True
        print(f'\n[+] ZeroDivisionError caught on iteration {i}!')
        break

    # The test passes if it successfully caught the expected bug
    self.assertTrue(exception_caught, "ZeroDivisionError was not triggered")

    def test_get_rand_negative_input(self):
        self.assertEqual(get_rand(-5), 1)

if __name__ == '__main__':
    unittest.main()
```

Question Number 5:

```
from Crypto.Cipher import AES

from Crypto.Util.Padding import pad, unpad

from base64 import b64encode, b64decode

from binascii import unhexlify

import os

# Current state of the encrypted payload and keys

source_encrypted =
'1EHor0VZBlmZyzL1HuLY4T/J79v6NPiUExoRCSfVJGraKpPKz22URaI6eO8wI84iqd4vpw7lenUiESi
mQujOLOnSKlGMz3GvK7gJGwNxnXY34jbokf/9SXSrC61oATBz7OhGrMDcW0UNA3ZLzO9bTAee7
hEBm8+yS2o9zf+HJ1kYrIE4rXhSS3hv97UUC6ZCyhoVMpu1z0lmNReW75sefP0oetbLofFkiMA7055y
JasDCn2nbcp3CWK9ySIX8zoW'

iv = '7bde5a0f3f39fd658efc45de143cbc94'

key_hex = '3e83b13d99bf0de6c6bde5ac5ca4ae68'

# Decrypt and Execute payload

#This will decrypt the 'source_encrypted' string using the the hardcoded key and IV.

try:

    cipher = AES.new(unhexlify(key_hex), AES.MODE_CBC, unhexlify(iv))

    decrypted_source = unpad(cipher.decrypt(b64decode(source_encrypted)), AES.block_size).decode('utf-
8')

    exec(decrypted_source)

except Exception:

    print("[*] Initializing self-mutating script...")

    decrypted_source = "print(' Payload Executed Successfully!')"

    exec(decrypted_source)

# Polymorphic Mutation

# Fresh set of cryptographic key and re-encrypts the exact same payload

new_key = os.urandom(16)
```



```

new_iv = os.urandom(16)

new_cipher = AES.new(new_key, AES.MODE_CBC, new_iv)

new_encrypted = b64encode(new_cipher.encrypt(pad(decrypted_source.encode(),
AES.block_size))).decode()

# Self modification

#The script targets its own file path and swaps the old values with the newly generated one
file_path = os.path.abspath(__file__)

with open(file_path, 'r') as f:
    content = f.read()

content = content.replace(source_encrypted, new_encrypted)
content = content.replace(iv, new_iv.hex())
content = content.replace(key_hex, new_key.hex())

#Overwrite the original file with the modified source code
with open(file_path, 'w') as f:
    f.write(content)

print("[+] Self-mutation complete! Please check the source code for the new encrypted values.")

```